

Q1. Short Answer 1-3 lines [2 Marks each] a. Consider the code section below. Do you see any problem with the code? If yes, rewrite the code to fix the problem.

```
void * thr_fn(void *arg) {
    sleep(1);
    int *num = arg;
    (*num)++;
}

int main() {
    pthread_t tid1, tid2;
    int i=10;
    pthread_create(&tid1, NULL, thr_fn, (void *)&i);
    pthread_create(&tid2, NULL, thr_fn, (void *)&i);
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);
}
```

Answer:

It will cause a race condition since the variable i is passed by reference to the thread function. Both threads may try to update the value at the same time. We need to add mutex lock in the thread function. **void * thr_fn(void *arg) {**

```
    pthread_mutex_lock(&mutex);
    sleep(1);
    int *num = arg;
    (*num)++;
    pthread_mutex_unlock(&mutex);
}
```

b. Take a look at the code below. In this code, will the parent process terminate? Justify your answer briefly.

```
void handler(int signum) {
    exit(1);
}

void main() {
    int cpid, status;
    sigset_t s;
    sigfillset(&s);
    sigprocmask(SIG_BLOCK, &s, NULL);
    signal(SIGUSR1, handler);
    if((cpid = fork()) == 0) {
```

```

printf("I am child");
while(1)
continue;
}
sigprocmask(SIG_UNBLOCK, &s, NULL);
print("I am parent");
kill(cpid, SIGUSR1);
wait(&status);
}

```

Answer:

The parent process does not terminate because the fork copies the set of signals blocked in the child process, so unblocking it only in the parent has no effect in handling signals in the child process. Thus, signaling the child has no effect, and, as the child is in an infinite loop, the parent will wait forever.

c. Take a look at the code below. In this code, will the parent process terminate? Justify your answer briefly.

```

void sig_kill_handler(int signum) {
    printf("I am being killed\n");
    while(1)
        continue;
}

void main() {
    int cpid, status;
    signal(SIGKILL, sig_kill_handler);
    if((cpid = fork()) == 0) {
        printf("Looping Forever\n");
        while(1)
            continue;
    }
    else {
        kill(cpid, SIGKILL);
        wait(&status);
    }
}

```

Answer:

The parent does terminate because the default behavior of SIGKILL cannot be changed. Thus, sending a SIGKILL to the child immediately kills it and the wait successfully finishes without blocking (for long at least).

d. If we are executing a.out executable with the following code, how many times will the "ls" command be executed for the code below? Why?

```

int main()
{
    fork(); // line 1

```

```

    int pid = fork(); // line 2
    if (pid==0) // line 3
        execlp("ls", "ls", NULL); // line 4
    fork(); // line 5
    execlp("ls", "ls", NULL); // line 6
}

```

Answer:

6 times

fork() at line 1 will create a child process so we have two processes (p and c)

fork() at line 2 fork() will be executed again by both the processes (p and c) to create a child of their own (p, pc1, c, cc1) resulting in 4 total processes.

"If" statement at line 3 will be true only for the child processes pc1 and cc1 (not for parent processes p and c) and both will execute "ls" using exec() at line 4 → 2 ls execution

Parent processes p and c will execute continue execution and will execute fork() at line5 resulting in two more child (p, pc2, c, cc2). Each of these 4 processes now executes exec system call at line 6 to execute "ls" hence, 4 times.

Hence total 6 "ls" will be executed.

e. Assume that file1.txt contains the data as shown below. Assuming that the parent process performs the file IO before the child process, what will the file content after the code's execution? Explain your answer briefly.

Content of file1.txt:

A course on System Programming

```

char buf1[100];
int fd1 = open("file1.txt", O_RDWR);
if (fork() > 0)
    read(fd1, buf1, 12);
else
    write(fd1, "Python", 6);

```

Answer:

fork() acts as a dup therefore reference count of fd1 will be 2 and the offset will change for both process. Parent reads the first 12 bytes moving file pointer at S (for System) and child process will write Python over System.

Hence the file content will be "A course on Python Programming class."

f. Explain in a sentence or two what the following code is doing.

```

int cnt=0;
void int_handler()
{
    cnt++;
}

int main(void)
{

```

```

        oldhandler = signal(SIGINT, int_handler);
        alarm(10);
        while(1);
    }

```

Answer:

Count how many times control+c was pressed within 10 seconds.

g. If we are executing a.out executable with the following code, how many times will the "a.out" command be executed for the code below? Why?

```

int main()
{
    int pid = fork(); // line 1
    if (pid>0) // line 2
        execl("./a.out", "./a.out", NULL); // line 3
    int pid = fork(); // line 4
    fork(); // line 5
    execl("./a.out", "./a.out", NULL); // line 6
}

```

Answer:

Infinitely calls a.out

h. Will the code below cause deadlock? Briefly explain your response.

```

int fd[2];
pthread_t tid1, tid2;
void *thr_fn(void *arg) {
    pthread_mutex_lock(&mymutex);
    write(fd[1], msg, strlen(msg));
}
int main() {
    pthread_mutex_t mymutex;
    pthread_mutex_init(&mymutex, NULL);
    pipe(fd);
    pthread_create (&tid1, NULL, thr_fn, NULL);
    pthread_create (&tid2, NULL, thr_fn, NULL);
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);
    ret = pthread_mutex_destroy(&mymutex);
}

```

Answer:

It will cause a deadlock because the first thread in execution has already acquired a lock on the mutex, which is not released when the second thread starts; hence, it will be waiting indefinitely.

i. What will be the output from the following code?

```

int main() {
    int sum;

```

```

for (i=0; i<3; i++) {
    if (fork() > 0) {
        int status;
        sum = sum + wait(&status);
    }
    else {
        int i = 10;
        exit(i);
    }
}
printf("%d", sum);
}

```

Answer:

Prints the summation of the 3 child's pids.

- j. We have the following content in the makefile where myapp.c uses functions defined in abc.c and def.c from the library. Find the errors in the makefile and rewrite the makefile with corrections.

```

CC=gcc
%.o: %.c
        $(CC) -o $@ $^
libmylib.so: abc.o def.o
        ar rs $@ $^
myapp.out: myapp.o libmylib.so
        $(CC) -o $@ $<

```

Answer:

```

CC=gcc
%.o: %.c
        $(CC) -o $@ $<
libmylib.so: abc.o def.o
        $(CC) -shared $@ $^
myapp.out: myapp.o libmylib.so
        $(CC) -o $@ $^

```

- k. Take a look at the code below. In this code, will the parent process terminate? Justify your answer briefly.

```

int main()
{
    int status;
    fork();
    wait(&status);
}

```

Answer:

Child process does not have any child process so wait will return -1 for child and so child will terminate. Since the child terminates the parent process will also come out of wait and will terminate.

I. Consider the thread_fn function as shown below. Assume that two threads are allowed to call the function, which controls the execution of the critical section part of the code using a shared variable. Do you see any problem with this code? Briefly explain the reason. If you see a problem, rewrite the code to fix it.

```
int shared=0;
void *thread_fn(void *arg) {
    if (shared == 0)
        shared = 1;
    else return;
    // critical section code goes here
    shared = 0;
}
```

Answer:

The problem is caused by race conditions with a shared variable when multiple threads execute the function since checking the value and setting the value for a shared variable is not atomic. We need to use a mutex instead of a shared variable.

```
Pthread_mutex_t m;
void *thread_fn(void *arg) {
    pthread_mutex_lock(m)
    // critical section code goes here
    pthread_mutex_unlock(m)
}
```

Q2. There are 2 processes with 2 threads each. A single pipe is shared between both processes to exchange the data and both the thread of each process generates a data to be written to a pipe. A partial code is provided below. The parent/child process will send SIGUSR1 to the child/parent when one of its threads wants to write to the pipe and SIGUSR2 when it is done. You also need to implement locking so that only one of the two threads of a writer process is allowed write to the pipe. You need to fill in the code so that only one thread of either parent or child process can write to the pipe. [6 Marks]

```
#define READ 0
#define WRITE 1
char msg[1000];
int fd[2];
void *writer(void *p)
{
```

```
    sem_wait(&mysem); // or pthread_mutex_lock(&mymutex)
```

```

        if (getpid() == parent_pid) {
            Kill(child_pid, SIGUSR1);
        }
    _____ else {
        Kill(parent_pid, SIGUSR1);

    }

```

```

// Assume that the function generates string data in msg global variable
write(fd[0][WRITE],msg,strlen(msg)+1);

```

```

        if (getpid() == parent_pid) {
            Kill(child_pid, SIGUSR2);
        }
    _____ else {
        Kill(parent_pid, SIGUSR2);

    }
    sem_post(&mysem); // or pthread_mutex_unlock(&mymutex)

```

```

}

```

```

int main(void)

```

```

{
    pipe(fd);
    parent_pid = getpid();
    child_pid = fork();
    pthread_t thread_id[2];

    _____ sem_t mysem; // or pthread_mutex_t mymutex
    sem_init(&mysem, 0, 1); // or pthread_mutex_init(&mymutex)

```

```

    pthread_create(&thread_id[0], NULL, writer, NULL);
    pthread_create(&thread_id[1], NULL, writer, NULL);
    pthread_join(thread_id[0], NULL);
    pthread_join(thread_id[1], NULL);

```

```

    _____ sem_destroy(&mysem) // or pthread_mutex_destroy(&mymutex)

```

Q3.

<pre> // Q3 a int i=2; int main(void) { int pid, childpid, status; i=i+3; // i=5 pid = fork(); i=i+4; // i=9 for both parent and child </pre>	<pre> // Q3 b int i=2; void function(int n) {i=i+n;} int main(void) { int pid, childpid, status; function(3); // i=5 </pre>
---	--

<pre> if (pid != 0) // parent process { childpid = wait(&status); i = i + WEXITSTATUS(status); // i=11+9=20 printf("parent: i = %d\n", i); // prints 20 } else // child process { i += 2; // i=11 for child printf("child: i = %d\n", i); // prints 11 exit(i); } </pre>	<pre> pid = fork(); function(4); // i=9 for both parent and child if (pid != 0) // parent process { childpid = wait(&status); function(WEXITSTATUS(status));// i=11+9=20 printf("parent: i = %d\n", i); // prints 20 } else // child process { function(2); // i=11 for child printf("child: i = %d\n", i); // prints 11 exit(i); } </pre>
---	---

- a. Review the code on the left column in the table below. State what values of "i" will be printed in parent and child processes. Explain your answer with reasoning. **[3 Marks] Answer:**

child: i=11

parent: i=20

- b. Now if we modify the code as shown in the right column in the table below. State if the output will change or not? Explain your answer with reasoning. **[3 Marks] Answer:**

Answer will not change because calling a function still updates the copy of a respective process. child: i=11

parent: i=20

Q4 Assume that the code below is executed as a program with both files having some data. For each row of the code section, provide details of changes in Process Open File Tables and System-Wide Open File Table. Changes to both tables are shown as an example when line 3 is executed to open file1.txt and return fd1. Please make sure to copy the required information from the previous row to the next row before you update. The last row of tables must have complete information. [10 Marks]

Line	Code	Process Open File Tables (OFT)	System-Wide Open File Table

1	char wrt_msg[10] = "123456789";	Index SysFD Ptr 1 fd1=10	SysFD Offset RefCnt inode Ptr 10 0 1 1000
2	char read_msg[10];	2 fd2=11	11 0 1 1000
3	int fd1 =	3	
4	open("file1.txt", "O_RDWR");	4	
	int fd2 = open("file1.txt",		

	"O_RDWR");		
5	write(fd1, wrt_msg,	Index SysFD Ptr 1 fd1=10	SysFD Offset RefCnt inode Ptr 10 0 1 1000
6	strlen(wrt_msg));	2 fd2=11	11 0 1 1000
7	lseek(fd1, -5,	3	11 0 3 7 1 1000
8	SEEK_END); write(fd2,	4	
9	read_msg, 3); read(fd2,		
	wrt_msg, 4);		
15	If (fork() > 0)	Parent Process	SysFD Offset RefCnt inode
16	{ int status;	OFT Index	Ptr
17	int fd4=open("file1.txt",	SysFD	10 5 4 2 1000
18	"O_RDWR");	Ptr	11 7 4 2 1000
19	read(fd4, read_msg,	1 fd1=10	12 0 5 1 1000
	5); wait(&status);	2 fd2=11	
	}	3 fd4=12	
		4	
10	else	Child Process	SysFD Offset RefCnt inode Ptr
11	{	OFT Index	10 5 10 4 2 3 1000
12	int fd3 = dup(fd1);	SysFD Ptr	11 7 1 1000
13	read(fd3, read_msg,	1 fd1=10	
14	5); }	2 fd2=11	
		3 fd3=10	
		4	

	Show Content of Final Tables	Parent Process OFT Index SysFD Ptr 1 fd1=10 2 fd2=11 3 fd4=12 4 Child Process OFT Index SysFD Ptr 1 fd1=10 2 fd2=11 3 fd3=10	SysFD Offset RefCnt inode Ptr 10 10 3 1000 11 7 1 1000 12 5 1 1000
--	---	--	--